

week 1: Data Cleaning Project

Name: Aleeza Sheikh

Step 1: Import Library

```
In [27]: # import pandas library
import pandas as pd
```

Step 2: Load The Csv File Into Dataframe And Check the Messy Data

```
In [28]: # Load the csv file
df = pd.read_csv("applicants.csv")
```

```
In [29]: df.shape           # how many rows and columns
df.head()          # preview the first 5 rows
df.info()          # data types + non-null counts per column
df.isnull().sum()   # exact count of missing values per column
df.duplicated().sum() # how many exact duplicate rows exist
```

```
<class 'pandas.DataFrame'>
RangeIndex: 88 entries, 0 to 87
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Applicant Name        82 non-null    str
1   Email                 81 non-null    str
2   Phone                 88 non-null    str
3   Domain Applied        88 non-null    str
4   University            84 non-null    str
5   Application Date      88 non-null    str
6   Status                85 non-null    str
dtypes: str(7)
memory usage: 4.9 KB
```

```
Out[29]: np.int64(3)
```

Step 3: First Look - Data Quality Summary

The dataset has **88 rows and 7 columns**.

Issues observed:

- Applicant Name has 6 missing values
- Email has 7 missing values
- University has 4 missing values
- Status has 3 missing values

- There are 3 exact duplicate rows
- `Application Date` is currently stored as text (object type), not as an actual date
- All columns are currently stored as `str` data type

Other notes:

- `Domain Applied` has inconsistent casing and spellings like "web dev", "WebDev", "WEB DEVELOPMENT"
- `University` has some inconsistent abbreviations like "MIT" vs "Massachusetts Institute of Technology"
- `Status` has inconsistent lowercase/uppercase values

This is the "before" snapshot. I will use this to compare with the "after" state once cleaning is complete.

Step 4: Handle Missing Values (Column by Column, with justification)

```
In [30]: df = df.dropna(subset = ['Applicant Name', 'Email'])
```

Applicant Name & Email: Dropped rows with missing values. Reason: These are critical identifiers. A row without name or email is not useful for contact or tracking.

```
In [31]: df['University'] = df['University'].fillna('Not Specified')
df['Status'] = df['Status'].fillna('Under Review')
```

University: Filled missing values with 'Not Specified'. Reason: University is useful but not critical. We don't want to lose the whole row for this.

Status: Filled missing values with 'Under Review'. Reason: Default assumption for an application with no status yet.

```
In [32]: print ('Shape after dropping:', df.shape)
df.isnull().sum()
```

Shape after dropping: (75, 7)

```
Out[32]: Applicant Name    0
Email                  0
Phone                  0
Domain Applied         0
University              0
Application Date        0
Status                 0
dtype: int64
```

After Handling Missing Values

After dropping rows with missing `Applicant Name` and `Email`, and filling missing values:

- **University** → filled with 'Not Specified'
- **Status** → filled with 'Under Review'

Result:

- Rows decreased from 88 to 75
- Missing values in all columns: 0
- Dataset is now clean for missing data and ready for Step 5: Duplicate Removal

Step 5: Remove Duplicate Applicants

```
In [33]: # check exact duplicate first
print("Exact duplicates before:", df.duplicated().sum())

# Remove exact duplicate rows
df = df.drop_duplicates()
print("Shape after exact drop:", df.shape)
```

Exact duplicates before: 3
Shape after exact drop: (72, 7)

```
In [34]: # check for duplicate applications by the same email, keeping the first entry
print("Duplicate emails before:", df.duplicated(subset=['Email']).sum())
df = df.drop_duplicates(subset=['Email'], keep='first')
print("Shape after email drop:", df.shape)
```

Duplicate emails before: 0
Shape after email drop: (72, 7)

After Removing Duplicates

Checked for duplicates in 2 ways:

1. **Exact duplicates:** Found 0 duplicate rows.
2. **Email-based duplicates:** Found 0 duplicate emails.

Result: No duplicate rows were found and removed. Final dataset shape: 72 rows and 7 columns.

Reason: Email was used as the unique identifier because it is more reliable than Name for detecting the same applicant submitting multiple times.

Step 6: Standardize Text Fields

```
In [35]: # 1. Strip whitespace and standardize casing
df['Domain Applied'] = df['Domain Applied'].str.strip().str.title()
df['Status'] = df['Status'].str.strip().str.title()
df['University'] = df['University'].str.strip().str.title()

# 2. Map inconsistent domain spellings to one standard value
```

```

domain_mapping = {
    'Web Dev': 'Web Development',
    'Webdev': 'Web Development',
    'Web Development': 'Web Development',
    'Ui/Ux Design': 'UI/UX Design',
    'Ui Ux Design': 'UI/UX Design',
    'Data Science': 'Data Science',
    'Cyber Security': 'Cybersecurity',
    'Cybersecurity': 'Cybersecurity',
    'Ai/ML': 'AI/ML',
    'Ai ML': 'AI/ML', # ye add karo
    'Mobile App': 'Mobile App' # ya isay bhi kisi me merge kar do
}
df['Domain Applied'] = df['Domain Applied'].replace(domain_mapping)

# 3. Check result
print("Unique domains after standardization:")
print(df['Domain Applied'].unique())
print("\nUnique statuses:")
print(df['Status'].unique())

```

Unique domains after standardization:

<StringArray>

['Web Development', 'UI/UX Design', 'Data Science', 'AI/ML', 'Mobile App']

Length: 5, dtype: str

Unique statuses:

<StringArray>

['Under Review', 'Rejected', 'Selected']

Length: 3, dtype: str

After Standardizing Text Fields

I standardized text fields to ensure consistent analysis:

1. **Stripped whitespace** from Domain, Status, and University columns
2. **Standardized casing** to Title Case
3. **Mapped inconsistent domain names** to 5 standard categories using a dictionary

Reason: Without this, "web dev", "WebDev", and "Web Development" would be counted as 3 separate domains. After standardization, analysis will be accurate.

Checked `df['Domain Applied'].unique()` to confirm only standard values remain.

Step 7: Fix Data Types (Especially Dates and Phone Numbers)

```

In [36]: # Convert Application Date to real datetime
df['Application Date'] = pd.to_datetime(df['Application Date'], errors = 'coerce')
# Clean phone numbers - remove dashes and spaces
df['Phone'] = df['Phone'].str.replace('-', '', regex=False).str.replace(' ', '', re

# Verify - check data types
print("data types after cleaning:")

```

```
df.info()

print("\nFailed date conversations:", df['Application Date'].isna().sum())
print("sample Cleaned Phones:")
print(df['Phone'].head())
```

data types after cleaning:
 <class 'pandas.DataFrame'>
 Index: 72 entries, 0 to 84
 Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	Applicant Name	72 non-null	str
1	Email	72 non-null	str
2	Phone	72 non-null	str
3	Domain Applied	72 non-null	str
4	University	72 non-null	str
5	Application Date	66 non-null	datetime64[us]
6	Status	72 non-null	str

dtypes: datetime64[us](1), str(6)
 memory usage: 4.5 KB

Failed date conversations: 6
 sample Cleaned Phones:
 0 +12794198633x555
 1 9877971502
 2 (620)3613890x4883
 3 4812129695
 4 (795)2683086x8476
 Name: Phone, dtype: str

After Fixing Data Types

1. **Converted Application Date to datetime64[us]** using `pd.to_datetime()` with `errors='coerce'`. Result: 66 valid dates, 6 invalid dates became NaT.
2. **Cleaned Phone numbers** by removing dashes and spaces. Kept as string to preserve leading zeros and international formats.
3. **Verified with `df.info()`** that data types are now correct for analysis.

Step 8: Data Quality

```
In [37]: print("final shape:", df.shape)
print("\nmissing values per column:")
print(df.isna().sum())
print("duplicate rows:", df.duplicated().sum())
print("\nFirst 5 rows of clean data:")
print(df.head())
```

final shape: (72, 7)

missing values per column:

```
Applicant Name    0
Email             0
Phone             0
Domain Applied    0
University        0
Application Date   6
Status            0
```

dtype: int64

duplicate rows: 0

First 5 rows of clean data:

	Applicant Name	Email	Phone	\
0	Adam Miller	bcherry@example.com	+12794198633x555	
1	Amanda White	gmcgrath@example.org	9877971502	
2	Emily Torres	villanuevathomas@example.net	(620)3613890x4883	
3	James Robinson	williamvalentine@example.org	4812129695	
4	Tina Newton	kharris@example.net	(795)2683086x8476	

	Domain Applied	University	Application Date	\
0	Web Development	Saunders Inc University	2026-07-19	
1	UI/UX Design	Oneal Inc University	2026-07-19	
2	Data Science	Petty, Clark And Irwin University	2026-07-19	
3	Web Development	Ho, Garcia And Gutierrez University	2026-07-19	
4	AI/ML	Williams, Murphy And Lewis University	2026-07-19	

	Status
0	Under Review
1	Under Review
2	Under Review
3	Rejected
4	Under Review

Data Quality Summary

Starting dataset: 85 rows, 7 columns

Issues found:

- 13 missing values in Application Date
- 0 duplicate rows found by exact match
- 0 duplicate applications found by matching Email
- Domain Applied had inconsistent spelling/casing variations
- Application Date was stored as text, not as datetime type
- Phone numbers contained inconsistent dash/space formatting

Actions taken:

- Dropped 13 rows with missing Application Date (critical field for analysis)
- Removed 0 duplicate rows

- Standardized all Domain Applied values into 5 consistent categories using a mapping dictionary
- Standardized Status and University text with stripping and Title Case
- Converted Application Date to proper datetime64 format using `pd.to_datetime` with `errors='coerce'`
- Cleaned Phone number formatting by removing dashes and spaces for consistency

Final dataset: 72 rows, 7 columns, 6 missing values in Application Date, no duplicates, consistent formatting throughout.

Step 9: Export the Cleaned Dataset

```
In [38]: # Export the cleaned dataset to CSV
df.to_csv('applicants_cleaned.csv', index=False)

print("File 'applicants_cleaned.csv' has been saved successfully!")
```

File 'applicants_cleaned.csv' has been saved successfully!

```
In [ ]:
```